shieldify



SMART CONTRACT SECURITY ASSESSMENT

# Pear Vault Upgrade

## SECURITY REVIEW

**28 OCTOBER 2026**

rt contracts, our shielding · Your smart contracts, our shielding · Your smart contracts, our shielding · Your smart contracts

# Contents


1 **About Shieldify** ..... 3

2 **Disclaimer** ..... 3

3 **About Pear Vault Upgrade** ..... 3

4 **Risk Classification** ..... 4

4.1 Impact ..... 4

4.2 Likelihood ..... 4

5 **Security Review Summary** ..... 4

5.1 Protocol Summary ..... 5

5.2 Scope ..... 5

6 **Findings Summary** ..... 5

7 **Findings** ..... 6

# 1 About Shieldify

Positioned as the first hybrid Web3 Security company, Shieldify shakes things up with a unique subscription-based auditing model that entitles the customer to unlimited audits within its duration, as well as top-notch service quality thanks to a disruptive 6-layered security approach. The company works with very well-established researchers in the space and has secured multiple millions in TVL across protocols, also can audit codebases written in Solidity, Vyper, Rust, Cairo, Move and Go.

Learn more about us at shieldify.org.

# 2 Disclaimer

This security review does not guarantee bulletproof protection against a hack or exploit. Smart contracts are a novel technological feat with many known and unknown risks. The protocol, which this report is intended for, indemnifies Shieldify Security against any responsibility for any misbehavior, bugs, or exploits affecting the audited code during any part of the project's life cycle. It is also pivotal to acknowledge that modifications made to the audited code, including fixes for the issues described in this report, may introduce new problems and necessitate additional auditing.

# 3 About Pear Vault Upgrade

Pear Vaults are managed pools of capital on HyperEVM, where depositors pool funds and a vault leader runs long/short strategies on Hyperliquid. Each vault is an ERC-4626 contract: depositors receive non-transferable shares that represent a pro-rata claim on the vault's net asset value (NAV), and deposits, withdrawals, NAV tracking and fee enforcement all run on HyperEVM.

Trading capital is bridged to HyperCore, where each vault has its own programmatic trading wallet that can place trades but cannot move funds out. A cash buffer is kept in the vault contract on HyperEVM so that withdrawals can be paid instantly; exits that the buffer cannot cover are queued until funds are bridged back.

This review covers an upgrade of the vault contracts. At a high level, the main functionalities are:

- Depositors deposit an asset token into a vault and receive ERC-4626 shares
- A factory deploys and registers vaults and can now create and fund a vault in a single transaction
- Deposits keep a configurable share of the vault's balance on HyperEVM as a cash buffer and bridge the excess to HyperCore

- Exits are paid instantly from the buffer when it has enough liquidity, or queued and settled after a bridge-back
- Each vault sets its own performance fee, fixed at the first deposit, and a withdrawal fee; the platform's fee recipient is read from a shared Comptroller

# 4 Risk Classification

| SEVERITY | IMPACT: HIGH | IMPACT: MEDIUM | IMPACT: LOW |
|---|---|---|---|
| **Likelihood: High** | CRITICAL | HIGH | MEDIUM |
| **Likelihood: Medium** | HIGH | MEDIUM | LOW |
| **Likelihood: Low** | MEDIUM | LOW | LOW |

## 4.1 Impact

- **High** – results in a significant risk for the protocol's overall well-being. Affects all or most users
- **Medium** – results in a non-critical risk for the protocol affects all or only a subset of users, but is still unacceptable
- **Low** – losses will be limited but bearable, and covers vectors similar to griefing attacks that can be easily repaired

## 4.2 Likelihood

- **High** – almost certain to happen and highly lucrative for execution by malicious actors
- **Medium** – still relatively likely, although only conditionally possible
- **Low** – requires a unique set of circumstances and poses non-lucrative cost-of-execution to rewards ratio for the actor

# 5 Security Review Summary

The security review lasted 3 days with a total of 48 hours dedicated to the audit by the Shieldify team.

Overall, the code is well-written. The audit report contributed by identifying one Medium, one Low and two Informational severity issues. They're mainly related to performance-fee configuration in the one-transaction vault creation flow, cash-buffer sizing during deposits, push-based fee transfers on exits and outdated NatSpec.

The Pear team has done a great job with their test suite and provided support and responses to all of the questions that the Shieldify researchers had.

## 5.1 Protocol Summary

| PROJECT NAME | PEAR VAULT UPGRADE |
|---|---|
| **Repository** | pear-vault-smartcontracts |
| **Type of Project** | ERC-4626 Delegated Trading Vaults |
| **Security Review Timeline** | 3 days |
| **Review Commit Hash** | 5d3ad281dda26fcdf7a910818ee5f3c4cf0f3fe6 |
| **Fixes Review Commit Hash** | 79c254699c4a80669fbba56cfd1eb3fcf2bebaf9 |

## 5.2 Scope

The review covers the vault upgrades implemented following our previous audit of Pear Vault. The changes comprise 262 net lines of code in `pear-vault-smartcontracts/src/`. The updated codebase is as follows:

| FILE | NSLOC |
|---|---|
| src/libraries/HyperliquidHelper.sol | 88 |
| src/hyperliquid/L1Read.sol | 264 |
| src/hyperliquid/ICoreWriter.sol | 5 |
| src/PearVault.sol | 751 |
| src/PearVaultFactory.sol | 138 |
| src/CoreWriterReadCaller.sol | 95 |
| src/Comptroller.sol | 268 |
| Total | 1609 |

# 6 Findings Summary

The following number of issues have been identified, sorted by their severity:

- **Medium** issues: 1
- **Low** issues: 1
- **Info** issues: 2

| ID | TITLE | SEVERITY | STATUS |
|---|---|---|---|
| [M-01] | `createVaultWithDeposit()` Permanently Locks the Performance Fee at Zero | MEDIUM | FIXED |
| [L-01] | Every Deposit Sweeps the Existing Cash Buffer to HyperCore, so Instant Exits Collapse Under Normal Flow and Any Depositor Can Empty the Buffer | LOW | FIXED |
| [I-01] | Every Fee-Bearing Exit Across All Vaults Reverts When feeRecipient Cannot Receive the Asset | INFO | ACKNOWLEDGED |
| [I-02] | Stale NatSpec Misstates Who Can Bootstrap a Vault, Who Reports Inbound Transfers and How Deposit Limits Are Keyed | INFO | FIXED |

# 7 Findings

## M-01

MEDIUM FIXED

### `createVaultWithDeposit()` Permanently Locks the Performance Fee at Zero

#### Description

`setPerformanceFeeBPS()` is owner-only and refuses once `firstDepositLoaded` is set, so investors can rely on the fee never changing after a vault is funded. `initialize()` never sets a fee, so every vault starts at zero and the leader must set the fee between creation and the first deposit.

PRs 15 and 16 added `PearVaultFactory.createVaultWithDeposit()`, which creates the vault and calls `loadFirstDeposit()` in the same transaction, and `loadFirstDeposit()` now accepts the factory as caller. That removes the window: by the time the leader controls the vault, `firstDepositLoaded` is already `true` and `_performanceFeeBPS` is permanently `0`. No role can recover it, because pearAdmin cannot call the setter either. The test suite does not catch this because every test that sets a performance fee uses the two-step `createVault()` then `loadFirstDeposit()` flow, while the `createVaultWithDeposit()` tests check only funding and refunds.

#### Location of Affected Code

File: src/PearVault.sol#L381-L386

```
function setPerformanceFeeBPS(uint256 feeBPS) external override onlyOwner {
  if (firstDepositLoaded) revert PerformanceFeeLocked();
  if (feeBPS > MAX_PERFORMANCE_FEE_BPS) revert FeeTooHigh();
```

```
    _performanceFeeBPS = feeBPS;
    emit PerformanceFeeBPSUpdated(feeBPS);
}
```

File: src/PearVault.sol#L462-L473

```
function loadFirstDeposit(
    uint256 amount
) external payable nonReentrant {
    // The factory may bootstrap on the leader's behalf so that creation and funding
    // are one transaction — a vault that cannot be funded is never created, instead
    // of being left on-chain unfunded. Shares, cost basis and the deposit limit still
    // key to `vaultOwner_`; only the payer and the caller differ.
    address vaultOwner_ = owner();
    address payer = msg.sender;
    if (payer != vaultOwner_ && payer != factory) revert NotAuthorized();
    if (firstDepositLoaded) revert FirstDepositAlreadyLoaded();
    if (totalSupply() != 0) revert InvalidAmount();
    // code
}
```

File: src/PearVault.sol#L492-L495

```
// `totalSupply() == 0 && firstDepositLoaded` as the forbidden price-reset state,
// and the bootstrap itself is legitimately in that shape until these shares exist.
firstDepositLoaded = true;
```

File: src/PearVaultFactory.sol#L129-L137

```
if (msg.value < requiredValue) revert InsufficientCreationFee();

vaultAddress = _createVault(owner_, agent, name_, description, imageURI, configId);

IERC20(asset).transferFrom(owner_, address(this), depositAmount);
IERC20(asset).approve(vaultAddress, depositAmount);
// Forward exactly what the bootstrap costs. The vault refunds any excess to ITS
// caller — this factory — so the excess is returned to the admin from here instead.
IPearVault(vaultAddress).loadFirstDeposit{ value: requiredValue }(depositAmount);
```

## Impact

Every vault created through the one-transaction path, which was added to replace the two-step flow in production, can never charge a performance fee. The leader loses all performance-fee revenue for the vault's lifetime and the platform loses its share of it. The only remedy is to create a new vault and ask investors to migrate. Investor funds are not at risk.

## Proof of Concept

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.30;

import {VaultFixture, PearVault} from "./VaultFixture.sol";
import {IPearVault} from "../src/interfaces/IPearVault.sol";

contract M01PerfFeeLockTest is VaultFixture {
  function test_M01_OneTxCreation_PermanentlyLocksPerfFeeAtZero() public {
    address a = makeAddr("agentOneTx");
    _mockAgent(a, 0);
    uint256 fee = comptroller.getCreationDepositFee() + comptroller.getAgentGasFee();

    vm.startPrank(owner);
    token.approve(address(factory), FIRST_DEPOSIT);
    PearVault v = PearVault(factory.createVaultWithDeposit{value: fee}(owner, a, "OneTx", "d", "u", 0, FIRST_DEPOSIT));
    assertTrue(v.firstDepositLoaded());
    assertEq(v.getPerformanceFeeBPS(), 0);

    vm.expectRevert(IPearVault.PerformanceFeeLocked.selector);
    v.setPerformanceFeeBPS(2000);
    vm.stopPrank();

    // pearAdmin cannot help either: owner-only.
    vm.prank(pearAdmin);
    vm.expectRevert();
    v.setPerformanceFeeBPS(2000);

    // Contrast: the two-step path leaves a window.
    address b = makeAddr("agentTwoStep");
    _mockAgent(b, 0);
    vm.startPrank(owner);
    PearVault w = PearVault(factory.createVault(owner, b, "TwoStep", "d", "u", 0));
    w.setPerformanceFeeBPS(2000);
    token.approve(address(w), FIRST_DEPOSIT);
    w.loadFirstDeposit{value: fee}(FIRST_DEPOSIT);
    vm.stopPrank();
    assertEq(w.getPerformanceFeeBPS(), 2000);
  }
}
```

Result:

```
[PASS] test_M01_OneTxCreation_PermanentlyLocksPerfFeeAtZero()
```

## Recommendation

Let the factory set the fee before the bootstrap and take it as a parameter of `createVaultWithDeposit()`:

```
// PearVault
function setPerformanceFeeBPS(uint256 feeBPS) external override {
  if (msg.sender != owner() && msg.sender != factory) revert NotAuthorized();
  if (firstDepositLoaded) revert PerformanceFeeLocked();
  if (feeBPS > MAX_PERFORMANCE_FEE_BPS) revert FeeTooHigh();
  _performanceFeeBPS = feeBPS;
  emit PerformanceFeeBPSUpdated(feeBPS);
}

// PearVaultFactory.createVaultWithDeposit(..., uint256 performanceFeeBps)
vaultAddress = _createVault(owner_, agent, name_, description, imageURI, configId);
IPearVault(vaultAddress).setPerformanceFeeBPS(performanceFeeBps);
```

## Team Response

Fixed.

L-01

LOW FIXED

## Every Deposit Sweeps the Existing Cash Buffer to HyperCore, so Instant Exits Collapse Under Normal Flow and Any Depositor Can Empty the Buffer

## Description

The vault keeps an EVM cash buffer "so that most withdrawals can be paid **instantly** without waiting on a HyperCore→EVM bridge", and a deposit is described as keeping `MIN_CASH_FLOW_BPS%` on EVM and bridging the excess (README §2). An exit is instant only if the buffer covers it; otherwise it is queued until funds are bridged back (README §4).

`_autoRebalanceToCore()` runs after every `deposit()` and `mint()`. It keeps `minCashFlowBPS` of the vault's **current** balance and bridges everything else, including the buffer that was already there. Each deposit therefore leaves only `bps × (buffer + deposit)` on EVM. At the default 10%:

1. **The buffer never scales with the vault.** Under normal deposit flow it converges to `bps / (10000 - bps)` of one deposit, about 11%. With the default 1,000 USDC per-address tier, that is about 111 USDC for a vault of any size, roughly 0.01% of a 1M vault. No setting other than 0 or 100% makes the buffer track the vault's size.
2. **Anyone can empty it on demand.** Each 10 USDC minimum deposit cuts the buffer tenfold: 100,000 → 10,001 → 1,001 → 101 → 11 USDC after four deposits. It costs little beyond gas, since the depositor keeps the shares, and it can be done at any time, including ahead of a known exit.

Once the buffer is swept, `redeem()` and `withdraw()` revert with `DirectWithdrawNotAllowed`, and `maxWithdraw()` / `maxRedeem()` drop to dust. Every holder must queue and wait for the leader or pearAdmin to bridge funds back. Requests already in the queue are unaffected, because `pendingWithdrawalAssets` is kept. With `minCashFlowBPS == 0`, auto-rebalancing is off and the issue does not arise.

**Not the documented design.** The comment on `_minCashFlowBPS` and `contract-reference.md` state the formula (keep `minCashFlowBPS` of the current balance, bridge the excess); the code note records that an earlier review item (I-05) corrected the comment's wording from "total deposits". None of the code, the docs, the known issues or the previous reports acknowledges the consequence: that the buffer is independent of the vault's size and can be drained by third-party deposits. A buffer of about 111 USDC cannot pay "most withdrawals" instantly on a vault of any meaningful size, which contradicts its stated purpose (README §2).

**Not the leader's accepted power (known issue L-9).** Known issue L-9 accepts that the leader may bridge the buffer with `transferToHyperCore()` "as leader-driven", and the leader also sets `minCashFlowBPS`. Here the leader makes the opposite choice, keeping a 10% buffer and never bridging manually, and still cannot keep it: ordinary deposits, or any untrusted depositor, sweep it, and no value of `minCashFlowBPS` other than 0 or 100% avoids this. The issue is also distinct from known issue L-3 (the queue reserve blocking instant exits) and known issue L-7 (fixed; the max views returning 0).

The `targetCash` formula is unchanged since baseline `6cb2507`. `_autoRebalanceToCore()` itself was modified in PR 15 (Felix skip removed), PR 19 (`_freeBalance()`) and PR 21 (activation-fee skip).

## Location of Affected Code

File: [src/PearVault.sol#L1306-L1339](src/PearVault.sol#L1306-L1339)

```
function _autoRebalanceToCore() internal {
  // code
  uint256 vaultBalance = _freeBalance();
  // code
  uint256 targetCash = (vaultBalance * _minCashFlowBPS) / 10000;
  uint256 mustKeep = targetCash > pendingWithdrawalAssets ? targetCash : pendingWithdrawalAssets;
  if (vaultBalance <= mustKeep) return;
```

```
    uint256 excessAmount = vaultBalance - mustKeep;  // includes the cash already in the vault
    // code
    _bridgeToCore(excessAmount);
}
```

File: src/PearVault.sol#L1272-L1274

```
function _canDirectWithdraw(uint256 amount) internal view returns (bool) {
    return amount <= _liquidForDirectExit();  // free balance minus the queue reserve
}
```

## Impact

Instant exits, the path the buffer exists for, are unreliable on every vault that uses auto-rebalancing. Under normal operation the buffer settles at about 11% of a single deposit, and any depositor can sweep it to HyperCore at will. All holders are then pushed into the queue, which pays out only after a bridge-back. No funds are lost or moved between holders.

**Why Low.** The trigger is cheap and frequent: ordinary deposits cause it, and any depositor can cause it at will. The consequence is limited to a temporary delay: no funds are at risk, queued requests stay protected, and queued exits are paid after a bridge-back. It is rated above Informational because it breaks documented, user-facing behaviour in normal operation and can be triggered by untrusted parties, not only by a trusted role.

## Recommendation

Bridge at most the new deposit's own excess, so that a deposit never sweeps cash already held. Pass `assets` from `_creditDeposit()` into `_autoRebalanceToCore()`:

```
uint256 excessAmount = vaultBalance - mustKeep;
uint256 depositExcess = assets - (assets * _minCashFlowBPS) / 10000;
if (excessAmount > depositExcess) excessAmount = depositExcess;
```

This makes the buffer track deposits rather than the current balance, a change of semantics the team should opt into deliberately. Alternatively, size the buffer against vault value, `max(_minCashFlowBPS * totalAssets() / 10000, pendingWithdrawalAssets)`, and reuse the NAV already computed during the deposit. Neither change affects the leader's ability under known issue L-9 to bridge with `transferToHyperCore()`.

## Team Response

Fixed.

I-01

INFO ACKNOWLEDGED

## Every Fee-Bearing Exit Across All Vaults Reverts When feeRecipient Cannot Receive the Asset

### Description

Every exit (`redeem()`, `withdraw()`, `executeWithdrawal()`, `batchExecuteWithdrawal()`) pays its fees in the same transaction: `_transferFees()` pushes the withdrawal fee and the platform's share of the performance fee to `comptroller().getFeeRecipient()`. If that address cannot receive the asset, for example because Circle blacklists it, the fee transfer reverts and the exit reverts with it. Every vault on a factory reads the recipient through `IPearVaultFactory(factory).comptroller()`, so one blocked address stops exits on all of them at once. While `minFeeAmount` is nonzero (1,000 units on mainnet), every exit pays a withdrawal fee: `_exitFees()` floors it at the minimum and caps it at the gross, and a zero-gross exit already reverts. Holders can still queue and cancel, but nothing settles until the owner sets a new recipient with `setFeeRecipient()`.

PR 19 already moved the leader's share of the performance fee to a pull model (`leaderFeesOwed` / `claimLeaderFees()`). The code gives the reason: otherwise "a Circle blacklist … would have frozen exits for every holder". The withdrawal fee and the platform's share, in the same function, still use the push model.

The two push transfers are functionally unchanged since baseline `6cb2507` (the recipient lookup was only inlined), and this behaviour is not covered by the known issues or the previous reports. The earlier Shieldify item L-11 concerned rounding in the fee split, not a recipient that cannot receive funds. The issue is filed as Informational because the recipient is a Pear-controlled address (on mainnet, Comptroller `0xD729A94d6366a4fEac4A6869C8b3573cEe4701A9` returns the same EOA, `0x105A15b7d2FDfA5037eC87d7889E76801F07733e`, for `getFeeRecipient()`, `owner()` and `getPearAdmin()`), no funds are lost, and a single `setFeeRecipient()` call restores every exit. The trigger, however, lies with a third party (the token issuer), and the outage affects all vaults at the same time.

### Location of Affected Code

File: src/PearVault.sol#L1589-L1606

```
function _transferFees(address user, uint256 withdrawalFee, uint256 performanceFee) internal {
  address feeRecipient = comptroller().getFeeRecipient();
  if (withdrawalFee > 0) {
    vaultToken().safeTransfer(feeRecipient, withdrawalFee);     // push, inside every exit
    // code
  }
  if (performanceFee > 0) {
    // code
```

```
        leaderFeesOwed += leaderAmount;                    // pull (PR 19), leader share only
        if (platformAmount > 0) {
          vaultToken().safeTransfer(feeRecipient, platformAmount);  // push, same exposure
        }
        // code
    }
}
```

## Impact

While the fee recipient cannot receive the asset, every exit on every vault that shares the Comptroller reverts. Funds are frozen, not lost, until the owner sets a new recipient with `setFeeRecipient()`.

## Recommendation

Accrue the withdrawal fee and the platform's share in a counter that `_freeBalance()` excludes from NAV, as PR 19 does with `leaderFeesOwed`, and let the recipient claim it separately. As with `leaderFeesOwed`, count the accrual in `totalWithdrawn` at settlement and not again at claim time. If the remaining EIP-170 headroom does not allow the change, document the behaviour as accepted, with `setFeeRecipient()` as the recovery path.

## Team Response

Acknowledged.

I-02

INFO FIXED

### Stale NatSpec Misstates Who Can Bootstrap a Vault, Who Reports Inbound Transfers and How Deposit Limits Are Keyed

## Description

Several NatSpec and inline comments in `PearVault.sol` and `IPearVault.sol` describe behaviour the code does not have. Integrators and future maintainers rely on these comments to understand who can move funds and how accounting is keyed, so incorrect ones invite wrong assumptions.

1. `loadFirstDeposit()` **is documented as owner-only, but the factory can also call it and pay for it.** Both the implementation and the interface say "Load first deposit by vault owner" / "Can only be called once by vault owner". These comments were accurate at baseline `6cb2507`, where the function reverted unless `msg.sender == owner()`. PRs 15 and 16 changed the body to accept `payer == factory` and to pull the asset from that payer, so that `createVaultWithDeposit()` can bootstrap a vault in one transaction, but left both comments unchanged; the inline comment inside the function already describes the factory path, so the NatSpec now contradicts its own function body. As a result, the documentation understates who can trigger a fund-pulling bootstrap: through `createVaultWithDeposit()` it is the factory, spending the leader's approval (see known issue G-7).

2. `inFlightFromCore` **is documented as set by the owner or agent, but only pearAdmin can set it.** The comment reads "Set by owner/agent when withdraw2 action is initiated on HyperCore". Its only setter, `reportInFlightFromCore()`, is restricted to pearAdmin (the permissionless `clearExpiredInFlightTransfers()` can only delete an expired record).

3. **A comment claims** `_convertToShares()` **checks** `firstDepositLoaded`**.** Inside `loadFirstDeposit()`, the comment explains the ordering by stating that "`_convertToShares` treats `totalSupply() == 0 && firstDepositLoaded` as the forbidden price-reset state". `_convertToShares()` never reads `firstDepositLoaded`, and its own comment says that guarding on that flag "was tried and rejected".

4. **Deposit-limit accounting is described as keyed to the funder, but it is keyed to the receiver.** The `userCostBasis` NatSpec says `userDepositAmount` "is keyed to the funder `msg.sender`", and `_settleWithdrawalFees()` calls it "funder-keyed … Kept separate from cost basis on purpose (different key)". Since deposits were re-keyed to the receiver (the change the code comments label M-3), `userDepositAmount` is credited to and checked against the receiver (`_creditDeposit()`, `_guardDeposit()`), so both mappings follow the share owner.

5. `reportInFlightFromCore()` **a rationale the implementation no longer has.** `IPearVault` still says it is admin-only "so an untrusted leader cannot report an unbacked amount and inflate NAV (H-1)". The reviewed changes rewrote the implementation NatSpec: the reported amount is not added to `totalAssets()`, and the restriction exists so that the leader "cannot freeze deposits at will". The interface copy was not updated.

Items 1 and 5 became incorrect as a result of the reviewed changes. Items 2–4 are unchanged since `6cb2507` and are included because they sit in in-scope files next to code those changes touched.

## Location of Affected Code

File: [src/PearVault.sol#L453-L471](src/PearVault.sol#L453-L471)

```
/**
 * @notice Load first deposit by vault owner
 * @dev Can only be called once by vault owner to load initial funds
 ...
```

```
 */
function loadFirstDeposit(
  uint256 amount
) external payable nonReentrant {
  // code
  address vaultOwner_ = owner();
  address payer = msg.sender;
  if (payer != vaultOwner_ && payer != factory) revert NotAuthorized();
```

File: src/interfaces/IPearVault.sol#L235-L240

```
/**
 * @notice Load first deposit by vault owner
 * @dev Can only be called once by vault owner to initialize vault with funds
 * @param amount Amount of asset tokens to deposit
 */
function loadFirstDeposit(uint256 amount) external payable;
```

File: src/PearVault.sol#L98-L100

```
/// @notice In-flight transfer from HyperCore to HyperEVM
/// @dev Set by owner/agent when withdraw2 action is initiated on HyperCore
InFlightTransfer public inFlightFromCore;
```

File: src/PearVault.sol#L1210-L1212

```
function reportInFlightFromCore(
  uint256 amount
) external onlyPearAdmin validAmount(amount) {
```

File: src/interfaces/IPearVault.sol#L296-L303

```
/**
 * @notice Report an in-flight transfer from HyperCore to HyperEVM
 * @dev Called by the PEAR ADMIN (not the vault owner) when initiating a withdraw2
 *    action on HyperCore. Admin-only so an untrusted leader cannot report an
 *    unbacked amount and inflate NAV (H-1).
 */
function reportInFlightFromCore(uint256 amount) external;
```

File: src/PearVault.sol#L492-L495

```
// L-04: set AFTER the mint. `_convertToShares` treats
// `totalSupply() == 0 && firstDepositLoaded` as the forbidden price-reset state,
// and the bootstrap itself is legitimately in that shape until these shares exist.
firstDepositLoaded = true;
```

File: src/PearVault.sol#L138-L140

```
/// @dev Distinct from `userDepositAmount` (which is keyed to the funder `msg.sender` for
///      deposit-limit enforcement). Cost basis must follow the shares, so it is credited to
```

File: src/PearVault.sol#L1548-L1549

```
// Deposit-limit accounting (funder-keyed): shrink proportionally so the limit frees up
// as shares leave. Kept separate from cost basis on purpose (different key).
```

## Impact

Documentation only. The comments misstate who may bootstrap and fund a vault, who reports inbound transfers and why, how the zero-supply branch behaves, and how deposit limits are keyed. No execution path is affected.

## Recommendation

Update the comments to match the code, for example:

- `loadFirstDeposit()` in `PearVault.sol` and `IPearVault.sol`: "Callable once, by the vault owner or the factory (`createVaultWithDeposit()`). The asset is pulled from the caller; shares, cost basis and the deposit limit are credited to `owner()`."
- `inFlightFromCore`: "Set by pearAdmin through `reportInFlightFromCore()`."
- The ordering comment in `loadFirstDeposit()`: remove the claim about `_convertToShares()`, or state that the flag is set after the mint for ordering only.
- `userCostBasis` NatSpec and `_settleWithdrawalFees()`: "keyed to the receiver (the share owner), like the cost basis."
- `reportInFlightFromCore()` in `IPearVault.sol`: copy the implementation's NatSpec (the amount is not added to `totalAssets()`; admin-only so the leader cannot freeze deposits).

## Team Response

Fixed.




**Your security partner, for the long term**

SHIELDIFY.ORG · X.COM/SHIELDIFYSEC · LINKEDIN